

目录

区间查询公共组件	1
SA: LCP、子串比较与匹配区间	2
用 map 统计插入子串的精确次数 (step2)	4
SAM: 后缀链接树上的动态 endpos	4
Parent 树 DP: 汇总方向与 DFS 区间	6
AC 自动机: fail 树、动态模式集与禁串 DP	8
动态开关模式, 流式查询	8
长度为 n、避开所有禁串的字符串数量	9
PAM: 回文后缀、fail 树与回文划分	10
fail 树 DP、端点加权	10
series link 加速最少回文划分	11
KMP 自动机与 border 树 DP	12
预处理 $O(1)$ 字符转移	12
Parent 树: 前缀次数、最长公共 border、受限 border	12
序列自动机: 子序列判定与不同子序列计数	13
Trie 上的广义 PAM	15

区间查询公共组件

本章的 SA、SAM、AC、PAM 补充接口共用下面的树状数组; SA 的所有 LCP 查询共用同一张静态 RMQ 表。两者均使用 0-based 下标和左闭右开区间, 避免在各应用中重复实现。

```
#include <bits/stdc++.h>
using namespace std;

struct BIT {
    int n;
    vector<long long> t; // 外部 0-based, 内部 1-based; 初始权重全为 0
    explicit BIT(int n_ = 0) : n(n_), t(n + 1) {}
    // 位置 p 增加 x, 允许负数用于撤销;  $O(\log n)$ 
    void add(int p, long long x) {
        assert(0 <= p && p < n);
        for (++p; p <= n; p += p & -p) t[p] += x;
    }
    long long sum(int r) const { //  $[0, r)$ 
        long long s = 0;
        for (; r; r -= r & -r) s += t[r];
        return s;
    }
    long long sum(int l, int r) const { return sum(r) - sum(l); } //  $[l, r)$ 
};

struct RMQ {
    vector<int> lg; //  $lg[i]=\text{floor}(\log_2(i))$ 
    vector<vector<int>> st; //  $st[k][i]$ : 从 i 开始、长  $2^k$  的区间最小值
```

```

RMQ() = default;
explicit RMQ(const vector<int>& a) {
    int n = (int)a.size();
    lg.resize(n + 1);
    for (int i = 2; i <= n; ++i) lg[i] = lg[i / 2] + 1;
    if (!n) return;
    st.assign(lg[n] + 1, a);
    for (int k = 1; k <= lg[n]; ++k)
        for (int i = 0; i + (1 << k) <= n; ++i)
            st[k][i] = min(st[k-1][i], st[k-1][i + (1 << (k-1))]);
}
int query(int l, int r) const { // 非空 [l,r)
    assert(0 <= l && l < r && r <= (int)st[0].size());
    int k = lg[r - l];
    // 两个长  $2^k$  的区间覆盖  $[l,r)$ , min 允许重叠:  $O(1)$ 
    return min(st[k][l], st[k][r - (1 << k)]);
}
};

```

BIT 要求 $0 \leq p < n$ 、 $0 \leq l < r \leq n$; $\text{sum}(r)$ 返回 $[0,r)$, $\text{sum}(l,r)$ 返回 $[l,r)$ 。允许负增量, 总权重须装入 long long。RMQ::query(l,r) 要求非空区间。树状数组操作为 $O(\log n)$; RMQ 预处理时间/空间 $O(n \log n)$, 查询 $O(1)$ 。

SA: LCP、子串比较与匹配区间

这是 string/08-Sa.md 中 suffix_array_and_lcp 的应用层, 不重复后缀数组和 LCP 的构造。子串统一表示为 (start, length), 其中 start 是原串中的 0-based 起点。

依赖 00 的 BIT/RMQ。要求 $0 \leq \text{start} \leq n$ 、 $0 \leq \text{length} \leq n - \text{start}$; 空串可用于判等和比较, 匹配区间/计数必须传非空子串。suffix_lcp 的两个起点均须小于 n, 相同起点直接返回后缀长度。

```

struct SuffixArraySubstring { int start, length; }; // 原串 0-based 起点、长度

```

```

struct SAQuery {
    int n;
    vector<int> sa_, rk; // sa_[排名]= 起点, rk[起点]= 排名
    RMQ rmq; // 建在 h 上, h[i]=LCP(sa_[i-1],sa_[i])
    BIT bit; // 起点权重存到对应的 SA 排名上

    explicit SAQuery(const string& s) : n((int)s.size()), bit(n) {
        auto [sa, h] = suffix_array_and_lcp(s);
        sa_ = std::move(sa);
        rk.resize(n);
        for (int i = 0; i < n; ++i) rk[sa_[i]] = i;
        rmq = RMQ(h);
    }
    // 两个完整后缀的 LCP,  $O(1)$ ; x,y 均须在  $[0,n)$ 
    int suffix_lcp(int x, int y) const {
        if (x == y) return n - x;
        int l = rk[x], r = rk[y];
        if (l > r) swap(l, r);
        return rmq.query(l + 1, r + 1); // h 的闭区间  $[l+1,r]$ 
    }
};

```

```

}
// 内容精确判等, 允许空串;  $O(1)$ 
bool equal(SuffixArraySubstring a, SuffixArraySubstring b) const {
    return a.length == b.length &&
        (!a.length || suffix_lcp(a.start, b.start) >= a.length);
}
// 返回  $-1/0/1$ ; 一串是另一串前缀时比较长度, 否则比较后缀排名
int compare(SuffixArraySubstring a, SuffixArraySubstring b) const {
    int len = min(a.length, b.length);
    if (!len || suffix_lcp(a.start, b.start) >= len)
        return (a.length > b.length) - (a.length < b.length);
    return rk[a.start] < rk[b.start] ? -1 : 1;
}
// 所有匹配起点在 SA 上连续; 返回排名闭区间,  $O(\log n)$ 
pair<int, int> matching_interval(SuffixArraySubstring a) const {
    assert(a.length > 0); // 原串中的非空子串
    int p = rk[a.start], l = 0, r = p;
    // 左侧 LCP >= 长度 的判定从假变真, 找第一个真
    while (l < r) {
        int m = (l + r) / 2;
        if (suffix_lcp(sa_[m], a.start) >= a.length) r = m;
        else l = m + 1;
    }
    int L = l;
    l = p; r = n - 1;
    // 右侧判定从真变假, 找最后一个真
    while (l < r) {
        int m = (l + r + 1) / 2;
        if (suffix_lcp(a.start, sa_[m]) >= a.length) l = m;
        else r = m - 1;
    }
    return {L, l}; // SA 闭区间
}
// 静态次数不依赖 BIT; 允许重叠出现
int static_occurrence_count(SuffixArraySubstring a) const {
    auto [l, r] = matching_interval(a);
    return r - l + 1;
}
// 精确内容键: (长度, 匹配区间左端点),  $O(\log n)$ ; 空串统一为  $(0, 0)$ 
pair<int, int> key(SuffixArraySubstring a) const {
    if (!a.length) return {0, 0};
    return {a.length, matching_interval(a).first};
}
void add_start_weight(int p, long long x) { bit.add(rk[p], x); } // x 为增量
// 仅统计已加权起点; 初始为 0, 闭区间转 BIT 的半开区间
long long matching_weight(SuffixArraySubstring a) const {
    auto [l, r] = matching_interval(a);
    return bit.sum(l, r + 1);
}
const vector<int>& sa() const { return sa_; }
const vector<int>& rank() const { return rk; }
};
using SuffixArrayApplications = SAQuery; // 保留旧调用名

```

预处理时间/空间 $O(n \log n)$ ，后缀 LCP、子串判等和比较为 $O(1)$ 。matching_interval 的二分判定只调用 $O(1)$ 的 LCP RMQ，所以总复杂度为 $O(\log n)$ 。静态出现次数为 $R-L+1$ ；动态权值查询在起点的 $rk[start]$ 上单点修改，再对 $[L,R]$ 求和，单次修改或查询为 $O(\log n)$ 。

用 map 统计插入子串的精确次数 (step2)

要统计的是“之前插入的子串中，有多少个与查询串内容和长度完全相同”，用上面的 key。同长度的相同子串拥有相同 SA 区间；反过来，同长度且左端点相同，它们就是那个后缀的同长度前缀。因此 (length,L) 是确定性内容键。不同位置的相同内容会合并；键只在同一份固定原串的 SA 内有效，不能跨两份 SA 比较。

```
struct SubstrCount {
    const SAQuery& sa; // sa 必须比计数器活得久，使用期间不能重建/移动
    map<pair<int,int>, long long> cnt;
    explicit SubstrCount(const SAQuery& a) : sa(a) {}

    // 插入一个指定起点、长度的子串；x=-1 可删除一次，允许一般权重
    void add(SuffixArraySubstring a, long long x = 1) {
        auto k = sa.key(a);
        auto& v = cnt[k];
        v += x;
        if (!v) cnt.erase(k);
    }
    // 只查完全相同的插入项；不存在时返回 0，不创建新的 map 项
    long long get(SuffixArraySubstring a) const {
        auto it = cnt.find(sa.key(a));
        return it == cnt.end() ? 0 : it->second;
    }
};
```

SubstrCount cnt(saq) 在 step2 循环外建一次；查询为 $cnt.get(\{pos[i].back(),d\})$ ，插入为 $cnt.add(\{pos[i][d-1],d\})$ 。这里 d 必须是插入/查询子串各自的真实长度，不能只传起点。同一轮应先查询再插入，避免把当前项统计为之前的项。清空时用 $cnt.cnt.clear()$ 。

若直接写 map，就是 $map<pair<int,int>, long long> cnt$ ，插入 $++cnt[saq.key(\{p,len\})]$ 、查询 $cnt[saq.key(\{q,len\})]$ ；注意 operator[] 的查询会把不存在的键也插入，封装中的 get 用 find 避免这一点。已算好的键可以缓存，之后单次 map 操作仅为 $O(\log(M+1))$ 。

不缓存键时，单次增量/查询为 $O(\log(n+1) + \log(M+1))$ ，map 额外空间为 $O(M)$ ，M 是当前非零权重的不同内容数。空串计数在这里仅来自显式插入空串，不自动等于原串的 $n+1$ 个边界。键的默认顺序先比较长度，不是子串字典序；需要字典序时仍用 $SAQuery::compare$ 。

不能用 LCP 恰好等于 len 替代插入长度等于 len。例如原串 $aba\#abb$ ，插入 {4,3} 即 abb ，查询 {0,2} 即 ab ：完整后缀的 LCP 恰好为 2，但精确计数必须返回 0。即使“LCP 至少 len 减去 LCP 大于 len”能正确统计 LCP 等于 len，也没有记录插入项长度。

三种计数必须区分：static_occurrence_count 是原串中的全部出现次数；matching_weight 是已加权起点的后缀前缀匹配；SubstrCount::get 才是插入子串多重集的精确次数。BIT 从未记录插入长度，不能在查询时推回它。

SAM：后缀链接树上的动态 endpos

这是 string/01-Sam.md 中 SuffixAutomaton 的应用层，依赖 00 的 BIT，不重复 extend 和克隆状态构造。pre[p] 即每个位置扩展后的 prefix_state[p]。字符为小写字母，子串定位/判等只接受正长度。树和倍增预处理完成后，不能继续 extend。

```

using SamSubstringKey = pair<int,int>; // (状态, 长度)

struct SAMQuery {
    SuffixAutomaton a;
    vector<int> pre, tin, tout; // pre[p]: 扩展第 p 个字符后的 last
    vector<vector<int>> up; // up[k][v]: 沿 suffix link 跳 2^k 次, 根跳到根
    BIT bit; // 在 tin[pre[p]] 存结束位置 p 的权重, 初始全 0

    explicit SAMQuery(string_view s) {
        for (char c : s) {
            a.extend(c);
            pre.push_back(a.last_state());
        }
        const auto& t = a.data();
        int m = (int)t.size(), lg = 1;
        while ((1LL << lg) <= m) ++lg;
        vector<vector<int>> g(m);
        up.assign(lg, vector<int>(m));
        // 反向 suffix link 建树; 不是 SAM 的字符转移图
        for (int v = 1; v < m; ++v) {
            up[0][v] = t[v].link;
            g[t[v].link].push_back(v);
        }
        for (int k = 1; k < lg; ++k)
            for (int v = 0; v < m; ++v)
                up[k][v] = up[k-1][up[k-1][v]];
        tin.resize(m); tout.resize(m);
        vector<int> st{0}, order;
        // 迭代 DFS 前序: 整棵子树占连续区间, 避免长链递归爆栈
        while (!st.empty()) {
            int v = st.back(); st.pop_back();
            tin[v] = (int)order.size();
            order.push_back(v);
            for (int u : g[v]) st.push_back(u);
        }
        // 逆 DFS 序汇总子树右端点, 子树为 [tin[v], tout[v])
        for (int i = m - 1; i >= 0; --i) {
            int v = order[i];
            tout[v] = max(tout[v], tin[v] + 1);
            if (v) tout[t[v].link] = max(tout[t[v].link], tout[v]);
        }
        bit = BIT(m);
    }
    // 子串用 0-based 右端点 r、正长度 len 表示; O(log n)
    int substring_state(int r, int len) const {
        assert(0 <= r && r < (int)pre.size() && 1 <= len && len <= r + 1);
        int v = pre[r];
        // 尽量向上跳, 停在 length[link[v]] < len <= length[v]
        for (int k = (int)up.size() - 1; k >= 0; --k)
            if (a.data()[up[k][v]].length >= len) v = up[k][v];
        return v;
    }
    // 同状态可以有多个长度, 精确键必须保留 len

```

```

SamSubstringKey substring_key(int r, int len) const {
    return {substring_state(r, len), len};
}
bool equal(int r1, int len1, int r2, int len2) const {
    return len1 == len2 && substring_key(r1, len1) == substring_key(r2, len2);
}
void add_end_weight(int p, long long x) { bit.add(tin[pre[p]], x); } // 增量
// 一个结束位置属于 endpos(v), 当且仅当它的前缀状态在 v 的子树内
long long state_endpos_weight(int v) const { return bit.sum(tin[v], tout[v]); }
long long substring_endpos_weight(int r, int len) const {
    return state_endpos_weight(substring_state(r, len));
}
const vector<int>& prefix_states() const { return pre; }
const SuffixAutomaton& sam() const { return a; }
};
using SuffixAutomatonApplications = SAMQuery; // 保留旧调用名

```

suffix link 树上, 状态 v 的 DFS 子树恰好覆盖 $\text{endpos}(v)$ 对应的所有前缀状态。因此给结束位置 p 加权时修改 $\text{tin}[\text{pre}[p]]$, 查询状态 v 时求 $[\text{tin}[v], \text{tout}[v])$ 。根状态查询返回全部已加权的结束位置之和, 不定义空串的 $n+1$ 次出现。

预处理时间/空间 $O(n \log n)$, 定位状态、两次定位后的判等、动态修改/查询均为 $O(\log n)$; 已取得 (状态, 长度) 键后的比较为 $O(1)$ 。只有倍增表把根的祖先设为根, 不改变原 SAM 的 $\text{link}[0]=-1$ 。

状态只确定一个 endpos 等价类; 同一状态可能代表多个长度不同、内容也不同的子串。故子串的确定性键必须是 (state, length)。

Parent 树 DP: 汇总方向与 DFS 区间

本篇是公共依赖。所有节点 0-based, 根的父亲约定为 -1, 输入必须是一棵非空有根树。复用时不要把自动机转移图当成 Parent 树:

结构	本篇所说的 parent	祖先的含义	每个原串位置挂在哪儿
SAM	suffix link	更短后缀的 endpos 类	扩展后的 last
AC	fail	当前已读文本的后缀, 同时为 Trie 前缀	扫描文本后的状态
PAM	fail	更短的回文后缀	add 返回的最长回文后缀
KMP	$\text{prefix}[\text{length}-1]$	当前前缀的 border	前缀长度 $i+1$

SAM 的转移是 DAG; AC 的 Trie 父边表示删去最后一个字符; PAM 的建树父边表示删去首尾两个字符。它们均不同于上述 suffix/fail 父边。PAM 的偶根指向奇根, 奇根的自环要改成 -1 才能交给树代码。

两个常见方向:

- 儿子向父亲: 先把结束位置的次数、权重、最早/最晚端点放到其状态, 逆序合并。状态的子树结果就是其出现位置的聚合信息。
- 父亲向儿子: 在终止状态放标记或权重, 正序继承。当前状态根路径上的和就是此刻结束的全部模式的权值; OR 标记则表示是否命中任意禁串。

```

#include <bits/stdc++.h>
using namespace std;

struct TreeOrder {
    vector<int> parent, order, tin, tout; // parent[root]=-1, order 为 DFS 前序

```

```

vector<vector<int>> children; // 只存从父到子的边
TreeOrder() = default;
// fa 必须描述一棵非空有根树;  $O(V)$  预处理, 子树为  $[tin, tout)$ 
explicit TreeOrder(vector<int> fa) : parent(std::move(fa)) {
    int n = (int)parent.size(), root = -1;
    children.resize(n); tin.resize(n); tout.resize(n);
    for (int v = 0; v < n; ++v) {
        if (parent[v] == -1) root = v;
        else children[parent[v]].push_back(v);
    }
    assert(root != -1);
    vector<int> st{root};
    // 栈式 DFS 保证子树连续; 这里不是 BFS
    while (!st.empty()) {
        int v = st.back(); st.pop_back();
        tin[v] = (int)order.size();
        order.push_back(v);
        for (int u : children[v]) st.push_back(u);
    }
    assert((int)order.size() == n);
    // 子树处理完再更新父亲, 右端点取子树中最大 tin 加一
    for (int i = n - 1; i >= 0; --i) {
        int v = order[i], p = parent[v];
        tout[v] = max(tout[v], tin[v] + 1);
        if (p != -1) tout[p] = max(tout[p], tout[v]);
    }
}

bool is_ancestor(int u, int v) const { // 包含 u==v
    return tin[u] <= tin[v] && tin[v] < tout[u];
}

// 儿子向父亲汇总; op 为可交换、可结合操作, 如 sum/min/max/OR。
// a 长度等于节点数, 初值为自身贡献; 无贡献处填相应单位元。
template<class T, class F>
vector<T> subtree_fold(vector<T> a, F op) const {
    for (int i = (int)order.size() - 1; i > 0; --i) {
        int v = order[i], p = parent[v];
        a[p] = op(a[p], a[v]);
    }
    return a;
}

// 父亲向儿子继承, 返回含自身的根路径聚合; 顺序为父在前、子在后
template<class T, class F>
vector<T> root_path_fold(vector<T> a, F op) const {
    for (int v : order)
        if (parent[v] != -1) a[v] = op(a[parent[v]], a[v]);
    return a;
}
};
using ParentTreeIndex = TreeOrder;

```

预处理与一次 DP 均为 $O(V)$, 空间 $O(V)$, 使用迭代 DFS。不能对 SAM 按状态编号倒序汇总, 克隆的编号可能大于儿子; 应按长度降序或用这里的树序。AC 可用逆 BFS, PAM 的 fail 总指向更早创建的节点, KMP 的父长度更小。

动态场景复用 00-range-query.md 的 BIT，有两种互为对偶的写法，见 AC/PAM 完整封装：

- 结束位置动态加权：add(tin[state], delta)，查询模式状态子树 [tin[v], tout[v]]。
- 模式动态加权：在模式子树做差分区间加 add(tin[v], delta)、add(tout[v], -delta)，文本状态查询 sum(tin[state]+1)。差分树状数组要开 V+1，容纳 tout==V。

限制结束位置在 [l,r]：把查询拆成截止 r 的贡献减去截止 l-1 的贡献，按端点扫描，每个位置只激活一次，用第一种写法查询。若要求长度为 len 的子串完整落在原串 [a,b]，结束位置范围应是 [a+len-1,b]，不是 [a,b]。离线复杂度 $O((n+q)\log V)$ 。

首次/末次出现用子树 min/max（无贡献节点初始化为 INT_MAX/-1）；多个串的归属集合可改成 bitset OR，但注意 bitset 合并时间与字符串数量有关。LCA 直接复用 graph/10-euler-tour-lca.md，不要再写一份树剖。

AC 自动机：fail 树、动态模式集与禁串 DP

依赖 string/02-ACam.md、00-range-query.md、04-parent-tree-tricks.md。仓库原 AC 文件保持不变；复制到题目中使用时，把下面这段只读接口放入 AhoCorasick 的 public：区域。原类的数据为 private，应用层必须通过这些接口访问；插入与构造算法不变。验证脚本也仅在内存中的源码副本接入这段接口。

```
// 放进原类 public：调用前必须 build()，c 为小写字母
int state_count() const { return (int)nodes.size(); }
int transition(int v, char c) const { return nodes[v].next[c-'a']; }
int failure(int v) const { return nodes[v].fail; }
const vector<int>& breadth_first_order() const { return bfs_order; } // 不含根
```

扫描到状态 v 时，一个模式此刻结束，当且仅当其终止节点在 v 的 fail 祖先链上。故只检查 v 自身是否为终止点会漏掉后缀模式，如模式 a,ba 扫到 ba 时两个都命中。静态全文出现次数直接调用已有 count_occurrences，其逆 BFS 已是 fail 树 DP，不再重复实现。

动态开关模式，流式查询

```
struct ACQuery {
    AhoCorasick ac;
    vector<int> id; // id[i]: 第 i 个模式的终止状态，重复模式可能相同
    TreeOrder tr;
    BIT bit; // fail 子树区间加、文本状态单点查，存的是差分
    int cur = 0; // 已读文本后缀对应的 AC 状态
    explicit ACQuery(const vector<string>& s) {
        for (const auto& p : s) {
            assert(!p.empty());
            id.push_back(ac.insert(p));
        }
        ac.build();
        int n = ac.state_count();
        vector<int> fa(n, -1);
        for (int v = 1; v < n; ++v) fa[v] = ac.failure(v);
        tr = TreeOrder(fa);
        bit = BIT(n + 1); // tout 可以等于 n
    }
    // 第 i 个模式权重增加 x：初始全为 0，+1 激活、-1 撤销
    void add_pattern_weight(int i, long long x) {
        int v = id[i];
```

```

        // v 是哪些状态的 fail 祖先，就影响哪些状态的匹配答案
        bit.add(tr.tin[v], x);
        bit.add(tr.tout[v], -x);
    }
    // 返回以这个新字符为右端点的匹配权重和，允许重叠：O(Log V)
    long long feed(char c) {
        cur = ac.transition(cur, c);
        return bit.sum(tr.tin[cur] + 1);
    }
    void reset_text() { cur = 0; } // 新文本从根开始，模式权重保留
};
using DynamicAhoPatterns = ACQuery;

```

加权与读入一个字符均为 $O(\log V)$ ；构造 $O(26V + \sum |pattern|)$ ，空间 $O(26V)$ 。权重总和须装入 long long。在文本读到一半时激活一个模式，仅影响之后的查询，不追溯修改已经返回的答案；继续匹配会保留激活前已读的文本后缀。每份独立文本开始前调用 `reset_text()`。

长度为 n 、避开所有禁串的字符串数量

把模式终止标记沿 fail 从父到子传播，再在（已生成长度，AC 状态）上 DP。接受状态应是“沿 fail 链存在终止点”，不能只标记 Trie 叶子。

```

// 生成长度 len 的字符串，字母表为前 sigma 个小写字母，答案模 mod
int count_avoiding_patterns(const vector<string>& s, int len, int sigma, int mod) {
    assert(len >= 0 && 1 <= sigma && sigma <= 26 && mod > 0);
    AhoCorasick ac;
    vector<int> id;
    for (const auto& p : s) id.push_back(ac.insert(p));
    ac.build();
    int n = ac.state_count();
    vector<char> bad(n); // 当前状态或任一 fail 祖先为禁串终点
    for (int v : id) bad[v] = true;
    // 正 BFS: 父亲的标记先算完，否则漏掉较短的后缀模式
    for (int v : ac.breadth_first_order()) bad[v] |= bad[ac.failure(v)];
    if (bad[0]) return 0; // 空禁串
    vector<int> dp(n), ndp(n); // dp[v]: 当前长度、停在 v 的合法串数
    dp[0] = 1 % mod;
    for (int i = 0; i < len; ++i) {
        fill(ndp.begin(), ndp.end(), 0);
        for (int v = 0; v < n; ++v)
            for (int c = 0; c < sigma; ++c) {
                int u = ac.transition(v, char('a' + c));
                // 已命中禁串的路径直接丢弃，不再参与后续转移
                if (!bad[u]) ndp[u] = (int)((long long)ndp[u] + dp[v]) % mod;
            }
        dp.swap(ndp);
    }
    long long ans = 0;
    for (int x : dp) ans = (ans + x) % mod;
    return (int)ans;
}

```

生成字母表是前 `alphabet_size` 个小写字母，模式仍允许使用任意小写字母。DP 为 $O(nV|\Sigma|)$ ，滚动空间 $O(V)$ ，另加 AC 构造。长度很大且状态少时可对合法状态转移矩阵快速幂；要求出现全部模式时可把终止

标记改成 fail 继承的模式 bitmask，再扩展 DP 的 mask 维度，复杂度会多出 2^k 。

“匹配了多少次”与“多少种模式至少出现过一次”不同：后者静态扫描后检查已有 `count_occurrences()[terminal[id]]>0` 即可；重复模式按 ID 还是按内容去重要先定口径。不要在每个文本位置无条件遍历整条 fail 链，`a,aa,...` 会使时间退化到文本长度乘模式最大长度。

PAM：回文后缀、fail 树与回文划分

依赖 `string/03-Pam.md`、`00-range-query.md`、`04-parent-tree-tricks.md`。原串位置 0-based；`pre[p]` 记录原模板 `add(text[p])` 的返回值。PAM 每个非根状态唯一对应一个回文串，不像 SAM 的状态对应一段长度区间。查询右端点必须在原串内，回文状态编号至少为 2。

fail 树 DP、端点加权

`cnt[v]=cnt[fail[v]]+1` 表示状态 `v` 的回文后缀数量。每个位置取 `cnt[pre[p]]` 就是以该位置结尾的回文串出现次数；再对位置求和得到全部回文出现次数（包含重叠）。将 `+1` 换成 `+length[v]` 就得到回文后缀长度和。

```
struct PAMQuery {
    PalindromicTree a;
    vector<int> pre, cnt; // pre[r]: 最长回文后缀状态; cnt[v]: 其回文后缀数
    vector<long long> len_sum; // 状态 v 的所有非空回文后缀长度之和
    TreeOrder tr;
    BIT bit;

    explicit PAMQuery(string_view s) {
        for (char c : s) pre.push_back(a.add(c));
        const auto& t = a.data();
        int m = (int)t.size();
        vector<int> fa(m, -1);
        // 奇根 0 的自环改成 -1; 偶根 1 仍以奇根为父亲
        for (int v = 1; v < m; ++v) fa[v] = t[v].fail;
        tr = TreeOrder(fa);
        cnt.resize(m); len_sum.resize(m);
        // fail 总指向先创建的节点; 两个虚根贡献均为 0
        for (int v = 2; v < m; ++v) {
            cnt[v] = cnt[t[v].fail] + 1;
            len_sum[v] = len_sum[t[v].fail] + t[v].length;
        }
        bit = BIT(m);
    }

    int ending_count(int r) const { return cnt[pre[r]]; } // 以 r 结尾, 包含重叠
    long long ending_length_sum(int r) const { return len_sum[pre[r]]; }
    void add_end_weight(int r, long long x) { bit.add(tr.tin[pre[r]], x); } // 增量
    // 状态 v 的回文出现权重和; 在 fail 子树汇总结束位置
    long long palindrome_weight(int v) const {
        assert(v >= 2);
        return bit.sum(tr.tin[v], tr.tout[v]);
    }

    pair<vector<int>, vector<int>> occurrence_extrema() const {
        int m = (int)a.data().size();
        // 无直接结束位置时先填 min/max 的单位元, 再汇总子树
        vector<int> lo(m, INT_MAX), hi(m, -1);
```

```

    for (int i = 0; i < (int)pre.size(); ++i) {
        lo[pre[i]] = min(lo[pre[i]], i);
        hi[pre[i]] = i;
    }
    lo = tr.subtree_fold(lo, [](int x, int y) { return min(x,y); });
    hi = tr.subtree_fold(hi, [](int x, int y) { return max(x,y); });
    return {lo, hi}; // 非根状态的最早/最晚右端点
}
// dp[i]: 长度 i 的前缀最少分成几段非空回文, dp[0]=0; O(n Log n)
vector<int> minimum_partition_counts() const {
    const auto& t = a.data();
    int n = (int)pre.size(), m = (int)t.size();
    // dif 为相邻 fail 长度差; sl 跳过连续相同 dif 的整组
    vector<int> dif(m), sl(m), best(m, n + 1), dp(n + 1, n + 1);
    for (int v = 2; v < m; ++v) {
        int f = t[v].fail;
        dif[v] = t[v].length - t[f].length;
        sl[v] = dif[v] == dif[f] ? sl[f] : f;
    }
    dp[0] = 0;
    for (int i = 1; i <= n; ++i)
        for (int v = pre[i-1]; t[v].length > 0; v = sl[v]) {
            int f = t[v].fail;
            // 本组最短回文长度为 length[sl[v]]+dif[v]
            best[v] = dp[i - t[sl[v]].length - dif[v]];
            // 同差值时合并其余候选; best[f] 可能是上一位置留下的缓存
            if (dif[v] == dif[f]) best[v] = min(best[v], best[f]);
            dp[i] = min(dp[i], best[v] + 1);
        }
    return dp; // best 跨位置保留, 不能每轮清零
}
const PalindromicTree& pam() const { return a; }
const vector<int>& prefix_states() const { return pre; }
};
using PalindromicApplications = PAMQuery;

```

构造后 ending_count/ending_length_sum 为 $O(1)$, 端点增量和状态权值查询为 $O(\log n)$, 首次/末次端点汇总为 $O(n)$ 。静态出现次数仍调用 `pam().occurrence_counts()`; 最大 $\text{length}[v] * \text{count}[v]$ 直接枚举非根状态即可。两个虚根不应算作回文串。

series link 加速最少回文划分

朴素转移是枚举当前位置的所有回文后缀 v , 用 $\text{dp}[i - \text{length}[v]] + 1$ 更新 $\text{dp}[i]$ 。若逐个走 fail, 全相同字符会退化到 $O(n^2)$ 。

$\text{dif}[v] = \text{length}[v] - \text{length}[\text{fail}[v]]$; 沿 fail 链把连续相同 dif 的状态归为一组, $\text{sl}[v]$ (series link) 跳到该组之外。组内回文长度形成等差序列。代码先取这一组最短回文对应的 dp , 再在 dif 相同时使用 $\text{best}[\text{fail}[v]]$ 合并其余候选。

best 必须跨位置保留: $\text{fail}[v]$ 不一定在当前轮被遍历到, 读取的是由周期关系保证正确的历史缓存, 不能每轮整体清零, 也不能不加证明把它改成任意带额外约束的 DP。根的 dif 置 0, 长度 1 的状态跳到偶根。

上述标准 series DP 为 $O(n \log n)$ 时间、 $O(n)$ 额外空间; 依据为 Rubinchik 与 Shur 的 EERTREE 论文。接口返回每个前缀的最少段数, 非空串最少切割次数为 $\text{dp}[n] - 1$, 空串切割次数为 0。

KMP 自动机与 border 树 DP

依赖 `string/07-Kmp.md`。使用固定小写字母表，状态是已匹配的模式前缀长度 $0..m$ ，包含完整匹配状态 m 。已有 KMP 搜索不重复实现。

预处理 $O(1)$ 字符转移

```
struct KMP {
    vector<array<int,26>> go; // go[i][c]: 已匹配 i 个字符，再读 c 后的新长度
    explicit KMP(const string& s) : go(s.size() + 1) {
        int n = (int)s.size();
        auto pi = prefix_function(s);
        // 必须算到状态 n; 完整匹配后的下一次转移仍要允许重叠
        for (int i = 0; i <= n; ++i)
            for (int c = 0; c < 26; ++c)
                if (i < n && s[i] == char('a' + c)) go[i][c] = i + 1;
                else if (i) go[i][c] = go[pi[i-1]][c]; // 复用已算好的失配状态
    }
    int transition(int v, char c) const { return go[v][c-'a']; }
    int pattern_length() const { return (int)go.size() - 1; }
};
using KmpAutomaton = KMP;
```

预处理时间/空间为 $O(26m)$ ，每次字符转移 $O(1)$ 。模式非空时，每次转移后若状态等于 m 就产生一次匹配；下一字符通过 `go[m]` 自动失配，因此保留重叠匹配。空模式只有状态 0 ，应把初始边界也计作一次匹配，合计文本长度加 1 。

自动机 DP 状态常为 (生成长度, matched)：统计避开单个模式的串时，禁止转移到 m ；统计恰好出现 k 次时增加计数维度，每次到 m 加一。多模式的同类完整计数代码见 `05-ac-applications.md`，不再复制一份 DP。

Parent 树：前缀次数、最长公共 border、受限 border

下面另依赖 `04-parent-tree-tricks.md`；LCA 复用 `graph/10-euler-tour-lca.md`。根 0 表示空前缀，节点 i 表示长度为 i 的前缀。

```
vector<int> kmp_parent_array(const string& s) {
    auto pi = prefix_function(s);
    vector<int> fa(s.size() + 1, -1);
    // 节点编号是前缀长度，父亲是其最长真 border 长度；根 0 的父亲为 -1
    for (int i = 1; i <= (int)s.size(); ++i) fa[i] = pi[i-1];
    return fa;
}

struct BorderQuery {
    TreeOrder tr;
    EulerTourLca lca;
    vector<long long> cnt;
    explicit BorderQuery(const string& s)
        : tr(kmp_parent_array(s)), lca(tr.children, {0}) {
        // 每个非空前缀挂一次结束位置；子树和就是该前缀的总出现次数
        vector<long long> a(s.size() + 1, 1);
        a[0] = 0;
        cnt = tr.subtree_fold(a, plus<long long>{});
        cnt[0] = (long long)s.size() + 1; // 空串按  $n+1$  个边界计数
    }
};
```

```

    }
    long long prefix_occurrences(int len) const { return cnt[len]; }
    int longest_common_border(int x, int y, bool proper = false) const {
        int v = lca.lca(x,y);
        // LCA 默认允许前缀本身; 若要求双方都是真 border, 必要时再跳父亲
        if (proper && v && (v == x || v == y)) v = tr.parent[v];
        return v;
    }
};
using KmpBorderQueries = BorderQuery;

// ans[i]: 长度 i 的前缀中, 满足 2*border 长度 <= i 的非空 border 数
vector<int> nonoverlapping_border_counts(const string& s) {
    auto pi = prefix_function(s);
    int n = (int)s.size(), j = 0;
    vector<int> dep(n + 1), ans(n + 1); // dep[j]: j 到根链上的非空节点数
    for (int i = 1; i <= n; ++i) dep[i] = dep[pi[i-1]] + 1;
    for (int i = 1; i < n; ++i) {
        // j 跨位置维护, 不要每次重新从 pi[i] 跳, 否则可能退化到平方
        while (j && s[i] != s[j]) j = pi[j-1];
        if (s[i] == s[j]) ++j;
        while (j > (i + 1) / 2) j = pi[j-1]; // 首尾两份不能重叠
        ans[i+1] = dep[j];
    }
    return ans;
}

```

KmpBorderQueries 预处理 $O(n \log n)$ 时间/空间, 次数与公共 border 查询 $O(1)$ 。若只要次数, 使用代码中的 subtree_fold 即可做到 $O(n)$, 无需建 LCA。真 border 查询若 LCA 等于某个输入, 必须跳一次父亲; 空前缀的真 border 不存在, 这里仍返回 0 作为哨兵。

nonoverlapping_border_counts 为 $O(n)$ 时间/空间。维护的 matched 是当前前缀最长、且首尾两段不重叠的 border; 答案是它在失配树上的深度。这里阈值随前缀长度单调变化, 且每步匹配长度最多加一, 沿 fail 回退总计线性。不能对每个前缀重新从 prefix[i] 开始跳, 也不能把此摊还结论套到任意乱序长度上限查询。

序列自动机：子序列判定与不同子序列计数

这里的“序列”指子序列, 允许跳过原串字符; 它不解决要求连续的字串问题。固定小写字母表, 状态 $p=0..n$ 表示下一次只能使用原串 text[p..n) 的字符。

next[p][c] 保存从 p 开始字符 c 的最早出现位置。转移后变成 next[p][c]+1, 严格递增, 所以转移图为 DAG。总取最早位置不会损失之后的可行选择。

```

#include <bits/stdc++.h>
using namespace std;

struct SeqAM {
    vector<array<int,26>> nxt; // nxt[p][c]: 位置 >= p 的最早 c, -1 表示不存在
    explicit SeqAM(const string& s) : nxt(s.size() + 1) {
        int n = (int)s.size();
        nxt[n].fill(-1); // 状态 n 已用完整串, 不能再取字符
        for (int i = n - 1; i >= 0; --i) {

```

```

        nxt[i] = nxt[i+1];
        nxt[i][s[i]-'a'] = i;
    }
}
// p 是下一次可选位置, 转移返回新的 p; 失败 -1 不能用于继续转移
int transition(int p, char c) const {
    int q = nxt[p][c-'a'];
    return q == -1 ? -1 : q + 1;
}
// 返回 0-based 匹配位置; 空模式返回空数组, 失败返回 nullopt
optional<vector<int>> earliest_embedding(string_view s) const {
    vector<int> pos;
    int p = 0;
    for (char c : s) {
        p = transition(p, c);
        if (p == -1) return nullopt;
        pos.push_back(p - 1);
    }
    return pos;
}
bool contains(string_view s) const { // 只判子序列存在性, 额外空间 O(1)
    int p = 0;
    for (char c : s) {
        p = transition(p, c);
        if (p == -1) return false;
    }
    return true;
}
int distinct_subsequence_count(int mod) const {
    assert(mod > 0);
    int n = (int)nxt.size() - 1;
    vector<int> dp(n + 1); // dp[i]: s[i..n) 的不同非空子序列内容数
    for (int i = n - 1; i >= 0; --i)
        for (int c = 0; c < 26; ++c) {
            int j = nxt[i][c];
            // 首字符 c 只选最早的 j; 1 对应单字符串, dp[j+1] 对应继续接
            if (j != -1) dp[i] = (int)(((long long)dp[i] + 1 + dp[j+1]) % mod);
        }
    return dp[0]; // 不含空子序列
}
};
using SubsequenceAutomaton = SeqAM;

```

构造时间/空间 $O(26n)$; 单模式判定/构造为 $O(|pattern|)$, `contains` 额外空间 $O(1)$, 返回匹配位置时结果数组占 $O(|pattern|)$; 不同子序列计数为 $O(26n)$ 时间、 $O(n)$ 额外空间。若要包含空子序列, 答案再加 1 后取模。`aaa` 的不同非空子序列仅有 `a,aa,aaa` 三种, 而下标选择有七种。

“每个字符只走向最早可用位置”使每种字符串内容只有唯一的自动机路径, 故按首字符分组的 `1+dp[next]` 不重复计数。普通下标 DAG 若把每个相同字符位置都连边, 会计算下标方案而非不同内容。

字母表很大时可改为每个值保存出现位置数组, 每步二分找下一个位置: 空间降为 $O(n)$, 单模式查询变为 $O(|pattern| \log n)$ 。不要在无题目需求时为大值域直接分配 n^* 值域表。

Trie 上的广义 PAM

输入已经建好的 Trie: 根为 0, $tr[u][c]$ 为儿子, 0 表示无边; 字符 c 为 $0..25$, 每个非根节点只有一个父亲。不展开根到每个终点的字符串。

$t[0]$ 为奇根, $t[1]$ 为偶根; $last[u]$ 是根到 Trie 节点 u 的字符串的最长回文后缀状态。全局每种非空回文串只有一个状态。

```
#include <bits/stdc++.h>
using namespace std;

struct TriePAM {
    struct Node {
        array<int, 26> ch{}, direct{}; // ch[c]: 在回文首尾加 c 后的状态
        int len = 0, fail = 0; // fail: 最长真回文后缀
    };
    vector<Node> t = vector<Node>(2); // 0 奇根 (len=-1), 1 偶根 (len=0)
    vector<int> last; // last[u]: Trie 根到 u 的最长回文后缀状态

    // s[0]=-1 是哨兵, s.back() 是新字符; p 是追加前的最长回文后缀。
    // direct[v][c] 是 v 内部前驱字符为 c 的最长真回文后缀, 无解为奇根。
    int extend(int p, int c, const vector<int>& s) {
        int i = (int)s.size() - 1;
        // 先检查 p 外侧字符; 失败后一次 direct 跳转就找到可扩展后缀
        if (s[i - t[p].len - 1] != c) p = t[p].direct[c];
        if (!t[p].ch[c]) {
            int v = (int)t.size();
            t.emplace_back();
            t[v].len = t[p].len + 2;
            // 单字符的 fail 为偶根; 其余是 c + 更短可扩展后缀 + c
            t[v].fail = p == 0 ? 1 : t[t[p].direct[c]].ch[c];
            int f = t[v].fail;
            // 除了紧邻 f 的前驱字符, 其余 direct 项都继承 f 的表
            t[v].direct = t[f].direct;
            t[v].direct[s[i - t[f].len]] = f; // i-len[f] 是后缀 f 前一位
            t[p].ch[c] = v;
        }
        return t[p].ch[c];
    }

    explicit TriePAM(const vector<array<int, 26>>& tr) : last(tr.size(), 1) {
        assert(!tr.empty());
        t[0].len = -1;
        vector<int> s{-1};
        vector<pair<int, int>> st{{0, 0}}; // (Trie 节点, 下一条待扫描字符边)
        while (!st.empty()) {
            int u = st.back().first, c = st.back().second;
            if (c == 26) {
                st.pop_back();
                if (u) s.pop_back(); // 只恢复路径; 全局 PAM 状态和转移保留
                continue;
            }
            ++st.back().second;
            int v = tr[u][c];
            if (!v) continue;
        }
    }
};
```

```

        s.push_back(c);
        // 从 Trie 父节点的 last 扩展, 不能沿用上一个兄弟分支的 last
        last[v] = extend(last[u], c, s);
        st.push_back({v,0});
    }
}

int size() const { return (int)t.size() - 2; } // 全局不同非空回文串数

// w[u] 是 Trie 结束节点权重; 返回每个回文状态的出现权重和。
// w[u]=1 时共享前缀只计一次; 根权重 w[0] 忽略。
vector<long long> count(const vector<long long>& w) const {
    assert(w.size() == last.size());
    vector<long long> cnt(t.size());
    for (int u = 1; u < (int)last.size(); ++u) cnt[last[u]] += w[u];
    // fail 总在当前状态之前创建, 逆编号即可把贡献传给全部回文后缀
    for (int v = (int)t.size() - 1; v >= 2; --v) cnt[t[v].fail] += cnt[v];
    return cnt; // 两个根的值不作字符串答案
}
};

```

构造时间/空间为 $O(26V)$, $size()$ 为 $O(1)$, 一次 $count$ 为 $O(V)$, 其中 V 是 Trie 节点数。每条 Trie 边最多新建一个回文状态; DFS 只恢复当前路径, 已建 PAM 状态不回滚。

$direct[v][c]$: 回文串 v 内部, 前一个字符为 c 的最长真回文后缀; 不存在时指向奇根 0。新状态的 $direct$ 表从 $fail$ 状态复制, 再修改一项。根的 $direct$ 表均初始化为 0。这样每次扩展只检查常数次数字符与跳转, 不需要沿 $fail$ 逐个尝试。

普通 PAM 的 $while(fail)$ 直接搬到 Trie 上会退化: 一条长 a 链, 每个节点挂一个 b 儿子, 每个分支可能重新跳整条链。这里的 $direct$ link 消除了该问题。

计数口径: 非根 $w[u]=1$ 时按 Trie 上不同结束节点计数, 共享前缀只计一次; 若按字典中每个字符串的每次出现计数, 先给终点加字符串重数, 再在 Trie 子树求和作为 $w[u]$ 。不要把这两种次数混用。动态端点权重可复用 04 的 $fail$ 树 DFS 序和 00 的 BIT。

技术依据: EERTREE 的 $direct$ link 与多版本构造, 固定字母表下直接复制 26 项即可。